

FastAPI: From Basics to Advanced

A Practical Guide to Building Modern APIs with Python

Compiled for Self-Study

July 2026

Table of Contents

Introduction

- Why FastAPI?
- Who This Book Is For
- How This Book Is Organized

Part I: Foundations

Chapter 1: Setting Up Your Environment

- 1.1 Installing Python and a Virtual Environment
- 1.2 Installing FastAPI and Uvicorn
- 1.3 Your First Application
- 1.4 Understanding ASGI

Chapter 2: Path Operations and Routing

- 2.1 HTTP Methods
- 2.2 Path Parameters
- 2.3 Query Parameters
- 2.4 Combining Path and Query Parameters

Chapter 3: Request Bodies and Pydantic Models

- 3.1 Why Pydantic?
- 3.2 Using a Model as a Request Body
- 3.3 Combining Body, Path, and Query Parameters
- 3.4 Nested Models
- 3.5 Field Validation with `Field`

Part II: Core Features

Chapter 4: Response Models and Status Codes

- 4.1 Declaring a Response Model
- 4.2 Status Codes
- 4.3 Multiple Response Models with a Union
- 4.4 Excluding Unset or Default Values

Chapter 5: Validation and Error Handling

- 5.1 Automatic Validation Errors
- 5.2 Raising HTTP Exceptions
- 5.3 Custom Exception Handlers
- 5.4 Overriding the Default Validation Handler

Chapter 6: Dependency Injection

- 6.1 What Is a Dependency?
- 6.2 Class-Based Dependencies
- 6.3 Sub-Dependencies
- 6.4 Dependencies with `yield` (Setup / Teardown)
- 6.5 Global Dependencies

Chapter 7: Middleware, CORS, and Static Files

- 7.1 Writing Custom Middleware
- 7.2 CORS (Cross-Origin Resource Sharing)
- 7.3 Serving Static Files
- 7.4 GZip Compression

Part III: Real-World Applications

Chapter 8: Authentication and Security

- 8.1 OAuth2 with Password Flow and JWT
- 8.2 API Key Authentication
- 8.3 Scopes and Role-Based Access

Chapter 9: Database Integration

- 9.1 SQLModel — The FastAPI-Native ORM
- 9.2 Async Databases with SQLAlchemy 2.0
- 9.3 Alembic for Migrations

Chapter 10: Async Programming in FastAPI

- 10.1 `def` vs `async def`
- 10.2 Calling External APIs Asynchronously
- 10.3 Running Concurrent Tasks

Chapter 11: Background Tasks

Chapter 12: WebSockets

Chapter 13: File Uploads and Forms

- 13.1 Uploading a Single File
- 13.2 Multiple Files
- 13.3 Form Data

Part IV: Advanced Topics & Production

Chapter 14: Testing FastAPI Applications

- 14.1 The Test Client
- 14.2 Testing POST Requests with a Body
- 14.3 Overriding Dependencies in Tests
- 14.4 Testing Async Endpoints
- 14.5 Running Tests

Chapter 15: Advanced Pydantic

- 15.1 Custom Validators
- 15.2 Model Config and Aliases
- 15.3 Settings Management with `pydantic-settings`
- 15.4 Computed Fields and Model Methods

Chapter 16: Project Structure and Best Practices

- 16.1 A Scalable Directory Layout
- 16.2 `APIRouter`
- 16.3 Best Practices Checklist

Chapter 17: Deployment

- 17.1 Production ASGI Servers
- 17.2 Dockerizing a FastAPI App
- 17.3 Reverse Proxy with Nginx
- 17.4 Health Checks and Observability

Chapter 18: Performance and Advanced Topics

- 18.1 Caching
- 18.2 Rate Limiting
- 18.3 Pagination
- 18.4 GraphQL Alongside REST (Strawberry)
- 18.5 Lifespan Events

Appendix A: Quick Reference Cheat Sheet

Appendix B: Further Learning

Closing Notes

Introduction

FastAPI is a modern, high-performance Python web framework for building APIs. It is built on top of **Starlette** (for the web parts) and **Pydantic** (for the data parts), and it takes full advantage of Python type hints to give you automatic data validation, serialization, and interactive documentation.

Why FastAPI?

- **Fast:** Performance on par with NodeJS and Go, thanks to Starlette and async support.
- **Fast to code:** Type hints enable editor autocompletion and fewer bugs.
- **Automatic docs:** Interactive API documentation (Swagger UI and ReDoc) generated for free.
- **Standards-based:** Built on open standards like OpenAPI and JSON Schema.
- **Easy to learn:** Designed to be intuitive, with excellent editor support.

Who This Book Is For

This book assumes you know basic Python (functions, classes, decorators) but does not assume any prior web framework experience. We progress from installing FastAPI and writing your first endpoint, all the way to authentication, databases, testing, background tasks, WebSockets, and production deployment.

How This Book Is Organized

The book is divided into four parts:

1. **Part I — Foundations:** environment setup, routing, parameters, and request/response bodies.
2. **Part II — Core Features:** validation, dependency injection, middleware, and error handling.
3. **Part III — Real-World Applications:** authentication, databases, background tasks, WebSockets, file uploads.
4. **Part IV — Advanced & Production:** testing, advanced Pydantic, project structure, deployment, and performance.

Each chapter includes runnable code examples. Type them out yourself rather than copy-pasting — it builds muscle memory.

Part I: Foundations

Chapter 1: Setting Up Your Environment

1.1 Installing Python and a Virtual Environment

FastAPI requires Python 3.8 or newer (3.10+ recommended). Always work inside a virtual environment.

```
python3 -m venv venv
source venv/bin/activate    # On Windows: venv\Scripts\activate
```

1.2 Installing FastAPI and Uvicorn

FastAPI itself is just the framework; you need an ASGI server such as **Uvicorn** to actually run it.

```
pip install "fastapi[standard]"
```

This installs FastAPI along with common extras: Uvicorn, Pydantic, python-multipart, and email-validator.

1.3 Your First Application

Create a file named `main.py`:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, FastAPI!"}
```

Run it with:

```
fastapi dev main.py
```

or directly with Uvicorn:

```
uvicorn main:app --reload
```

Visit `http://127.0.0.1:8000` in your browser, and check out the automatically generated docs at:

- `http://127.0.0.1:8000/docs` (Swagger UI)
- `http://127.0.0.1:8000/redoc` (ReDoc)

1.4 Understanding ASGI

FastAPI is built on **ASGI** (Asynchronous Server Gateway Interface), the successor to WSGI. ASGI allows FastAPI to handle asynchronous code natively, which is what enables high concurrency for I/O-bound workloads (database calls, external API calls, file I/O).

Chapter 2: Path Operations and Routing

2.1 HTTP Methods

FastAPI provides a decorator for each HTTP verb:

```
@app.get("/items")
@app.post("/items")
@app.put("/items/{item_id}")
@app.patch("/items/{item_id}")
@app.delete("/items/{item_id}")
```

Each corresponds to a standard CRUD-style operation: **GET** (read), **POST** (create), **PUT** (replace), **PATCH** (partial update), **DELETE** (remove).

2.2 Path Parameters

Path parameters are declared using curly braces in the route and typed function arguments:

```
@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

Because `item_id` is typed as `int`, FastAPI automatically:

- Parses and converts the value from the URL string.
- Validates that it really is an integer (returns HTTP 422 if not).
- Documents it correctly in the OpenAPI schema.

Order Matters

When you have both a fixed path and a parameterized path, declare the fixed one first:

```
@app.get("/users/me")
def read_current_user():
    return {"user": "current"}

@app.get("/users/{user_id}")
def read_user(user_id: str):
    return {"user_id": user_id}
```

If `/users/{user_id}` came first, a request to `/users/me` would match it and treat `"me"` as a `user_id`.

Predefined Values with Enums

```
from enum import Enum

class ModelName(str, Enum):
    alexnet = "alexnet"
    resnet = "resnet"
    lenet = "lenet"

@app.get("/models/{model_name}")
def get_model(model_name: ModelName):
    return {"model_name": model_name}
```

2.3 Query Parameters

Any function parameter not part of the path is treated as a query parameter:

```
@app.get("/items")
def list_items(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}
```

A request to `/items?skip=20&limit=5` returns `{"skip": 20, "limit": 5}`. Parameters with default values are optional; those without defaults are required.

Optional Parameters

```
from typing import Optional

@app.get("/items/{item_id}")
def read_item(item_id: int, q: Optional[str] = None):
    if q:
        return {"item_id": item_id, "q": q}
    return {"item_id": item_id}
```

In modern Python (3.10+) you can also write `q: str | None = None`.

2.4 Combining Path and Query Parameters

```
@app.get("/users/{user_id}/items/{item_id}")
def read_user_item(
    user_id: int,
    item_id: str,
    q: Optional[str] = None,
    short: bool = False,
):
    item = {"item_id": item_id, "owner_id": user_id}
    if q:
        item.update({"q": q})
    if not short:
        item.update({"description": "A long description"})
    return item
```

Chapter 3: Request Bodies and Pydantic Models

3.1 Why Pydantic?

Pydantic models let you declare the “shape” of incoming JSON data using ordinary Python classes and type hints. FastAPI uses these declarations to validate, parse, and document request bodies automatically.

```
from pydantic import BaseModel
from typing import Optional

class Item(BaseModel):
    name: str
    description: Optional[str] = None
    price: float
    tax: Optional[float] = None
```

3.2 Using a Model as a Request Body

```
@app.post("/items")
def create_item(item: Item):
    item_dict = item.model_dump()
    if item.tax is not None:
        item_dict["price_with_tax"] = item.price + item.tax
    return item_dict
```

FastAPI reads the body as JSON, validates the data against `Item`, converts types as needed (e.g. string `"10.5"` to float), and gives you a fully-typed Python object — with editor autocompletion.

3.3 Combining Body, Path, and Query Parameters

FastAPI is smart about where each parameter comes from, based on its declaration:

```
@app.put("/items/{item_id}")
def update_item(item_id: int, item: Item, q: Optional[str] = None):
    result = {"item_id": item_id, **item.model_dump()}
    if q:
        result.update({"q": q})
    return result
```

- Parameters that match a path segment → **path parameter**.
- Singular types (`int`, `str`, `bool`, etc.) → **query parameter**.
- Parameters declared as a Pydantic model → **request body**.

3.4 Nested Models

```
class Image(BaseModel):
    url: str
    name: str

class Item(BaseModel):
    name: str
    price: float
    tags: list[str] = []
    image: Optional[Image] = None
```

Pydantic handles arbitrarily nested models, lists of models, and dictionaries, validating the whole structure recursively.

3.5 Field Validation with `Field`

```
from pydantic import BaseModel, Field

class Item(BaseModel):
    name: str = Field(..., min_length=3, max_length=30)
    price: float = Field(..., gt=0, description="Price must be positive")
    tags: list[str] = Field(default_factory=list)
```

`Field` lets you add constraints (`gt`, `ge`, `lt`, `le`, `min_length`, `max_length`, `regex`) and metadata used in the generated OpenAPI docs.

Part II: Core Features

Chapter 4: Response Models and Status Codes

4.1 Declaring a Response Model

You can restrict and shape what a path operation returns using `response_model`, independent of the internal data structure:

```
class UserIn(BaseModel):
    username: str
    password: str
    email: str

class UserOut(BaseModel):
    username: str
    email: str

@app.post("/users", response_model=UserOut)
def create_user(user: UserIn):
    return user
```

Even though `user` contains a `password`, the response is filtered down to only the fields declared on `UserOut` — a simple, declarative way to avoid leaking sensitive data.

4.2 Status Codes

```
from fastapi import status

@app.post("/items", status_code=status.HTTP_201_CREATED)
def create_item(item: Item):
    return item
```

Using the `status` module's named constants is preferred over magic numbers for readability.

4.3 Multiple Response Models with a Union

```
from typing import Union

class PlaneItem(BaseModel):
    description: str
    size: float

class CarItem(BaseModel):
    description: str
    brand: str

@app.get("/items/{item_id}", response_model=Union[PlaneItem, CarItem])
def read_item(item_id: str):
    ...
```

4.4 Excluding Unset or Default Values

```
@app.get("/items/{item_id}", response_model=Item, response_model_exclude_unset=True)
def read_item(item_id: str):
    return items[item_id]
```

This is useful for PATCH-style partial updates, where you only want to return fields that were explicitly set.

Chapter 5: Validation and Error Handling

5.1 Automatic Validation Errors

When incoming data fails validation, FastAPI automatically returns an HTTP 422 response with a detailed JSON body describing exactly which fields failed and why — no extra code required.

5.2 Raising HTTP Exceptions

```
from fastapi import HTTPException

items = {"foo": "The Foo Wrestlers"}

@app.get("/items/{item_id}")
def read_item(item_id: str):
    if item_id not in items:
        raise HTTPException(status_code=404, detail="Item not found")
    return {"item": items[item_id]}
```

You can attach custom headers too:

```
raise HTTPException(
    status_code=404,
    detail="Item not found",
    headers={"X-Error": "There goes my error"},
)
```

5.3 Custom Exception Handlers

```
from fastapi import Request
from fastapi.responses import JSONResponse

class UnicornException(Exception):
    def __init__(self, name: str):
        self.name = name

@app.exception_handler(UnicornException)
def unicorn_exception_handler(request: Request, exc: UnicornException):
    return JSONResponse(
        status_code=418,
        content={"message": f"Oops! {exc.name} did something wrong."},
    )
```

5.4 Overriding the Default Validation Handler

```
from fastapi.exceptions import RequestValidationError
from fastapi.responses import PlainTextResponse

@app.exception_handler(RequestValidationError)
def validation_exception_handler(request: Request, exc: RequestValidationError):
    return PlainTextResponse(str(exc), status_code=400)
```

Chapter 6: Dependency Injection

6.1 What Is a Dependency?

A **dependency** is a callable (usually a function) that FastAPI runs before your path operation, and whose return value is injected into your function's parameters. Dependencies are FastAPI's mechanism for sharing logic — database sessions, authentication, pagination, rate-limiting — across many endpoints.

```
def common_parameters(q: Optional[str] = None, skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

@app.get("/items")
def list_items(common: dict = Depends(common_parameters)):
    return commons

@app.get("/users")
def list_users(common: dict = Depends(common_parameters)):
    return commons
```

6.2 Class-Based Dependencies

```
class CommonQueryParams:
    def __init__(self, q: Optional[str] = None, skip: int = 0, limit: int = 100):
        self.q = q
        self.skip = skip
        self.limit = limit

@app.get("/items")
def list_items(common: CommonQueryParams = Depends(CommonQueryParams)):
    return commons
```

6.3 Sub-Dependencies

Dependencies can themselves depend on other dependencies, and FastAPI resolves the whole graph, caching results within a single request by default:

```
def query_extractor(q: Optional[str] = None):
    return q

def query_or_cookie_extractor(
    q: str = Depends(query_extractor),
    last_query: Optional[str] = Cookie(default=None),
):
    if not q:
        return last_query
    return q
```

6.4 Dependencies with `yield` (Setup / Teardown)

This pattern is the standard way to manage resources like database sessions:

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/items")
def read_items(db: Session = Depends(get_db)):
    return db.query(Item).all()
```

Code after `yield` always runs after the response has been sent (or if an exception occurs), guaranteeing cleanup.

6.5 Global Dependencies

```
app = FastAPI(dependencies=[Depends(verify_token)])
```

This applies a dependency to every path operation in the application — useful for things like API-key checks.

Chapter 7: Middleware, CORS, and Static Files

7.1 Writing Custom Middleware

Middleware wraps every request/response, useful for logging, timing, or adding headers:

```
import time
from fastapi import Request

@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start_time = time.time()
    response = await call_next(request)
    response.headers["X-Process-Time"] = str(time.time() - start_time)
    return response
```

7.2 CORS (Cross-Origin Resource Sharing)

Browsers block cross-origin requests unless the server explicitly allows them. FastAPI ships with `CORSMiddleware` :

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://example.com"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

7.3 Serving Static Files

```
from fastapi.staticfiles import StaticFiles

app.mount("/static", StaticFiles(directory="static"), name="static")
```

7.4 GZip Compression

```
from fastapi.middleware.gzip import GZipMiddleware

app.add_middleware(GZipMiddleware, minimum_size=1000)
```


Chapter 8: Authentication and Security

8.1 OAuth2 with Password Flow and JWT

The most common pattern for API authentication is issuing a **JWT (JSON Web Token)** after verifying a username/password, then requiring that token on subsequent requests.

Install the extra dependencies:

```
pip install "python-jose[cryptography]" "passlib[bcrypt]"
```

Hashing Passwords

```
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)

def get_password_hash(password: str) -> str:
    return pwd_context.hash(password)
```

Creating a JWT

```
from datetime import datetime, timedelta
from jose import jwt

SECRET_KEY = "a-very-secret-key"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or timedelta(minutes=15))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

The Login Endpoint

```
from fastapi.security import OAuth2PasswordRequestForm

@app.post("/token")
def login(form_data: OAuth2PasswordRequestForm = Depends()):
    user = authenticate_user(form_data.username, form_data.password)
    if not user:
        raise HTTPException(status_code=400, detail="Incorrect username or password")
    access_token = create_access_token(
        data={"sub": user.username},
        expires_delta=timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES),
    )
    return {"access_token": access_token, "token_type": "bearer"}
```

Protecting Endpoints

```

from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username = payload.get("sub")
        if username is None:
            raise HTTPException(status_code=401, detail="Invalid credentials")
    except jwt.JWTError:
        raise HTTPException(status_code=401, detail="Invalid credentials")
    return get_user(username)

@app.get("/users/me")
def read_users_me(current_user=Depends(get_current_user)):
    return current_user

```

8.2 API Key Authentication

For simpler service-to-service auth:

```

from fastapi.security import APIKeyHeader

api_key_header = APIKeyHeader(name="X-API-Key")

def verify_api_key(api_key: str = Depends(api_key_header)):
    if api_key != "expected-secret-key":
        raise HTTPException(status_code=403, detail="Invalid API Key")

```

8.3 Scopes and Role-Based Access

OAuth2 scopes let a token carry fine-grained permissions (e.g. `items:read`, `items:write`), which you declare per-endpoint and validate inside your dependency — useful for role-based access control in larger applications.

Chapter 9: Database Integration

9.1 SQLAlchemy — The FastAPI-Native ORM

SQLModel, created by the same author as FastAPI, combines SQLAlchemy and Pydantic into a single model definition.

```
pip install sqlmodel
```

```
from sqlmodel import SQLModel, Field, create_engine, Session, select

class Hero(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: Optional[int] = None

engine = create_engine("sqlite:///database.db")
SQLModel.metadata.create_all(engine)
```

A CRUD API with SQLModel

```
def get_session():
    with Session(engine) as session:
        yield session

@app.post("/heroes", response_model=Hero)
def create_hero(hero: Hero, session: Session = Depends(get_session)):
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero

@app.get("/heroes", response_model=list[Hero])
def read_heroes(session: Session = Depends(get_session)):
    return session.exec(select(Hero)).all()

@app.get("/heroes/{hero_id}", response_model=Hero)
def read_hero(hero_id: int, session: Session = Depends(get_session)):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero
```

9.2 Async Databases with SQLAlchemy 2.0

For high-concurrency applications, use SQLAlchemy's async engine with a driver like `asyncpg` (PostgreSQL):

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession

engine = create_async_engine("postgresql+asyncpg://user:pass@localhost/db")

async def get_session():
    async with AsyncSession(engine) as session:
        yield session

@app.get("/heroes")
async def read_heroes(session: AsyncSession = Depends(get_session)):
    result = await session.execute(select(Hero))
    return result.scalars().all()
```

9.3 Alembic for Migrations

Use **Alembic** to version-control schema changes:

```
pip install alembic
alembic init migrations
alembic revision --autogenerate -m "create heroes table"
alembic upgrade head
```

Chapter 10: Async Programming in FastAPI

10.1 `def` VS `async def`

FastAPI supports both regular and asynchronous path operation functions:

```
@app.get("/sync")
def sync_endpoint():
    return {"type": "sync"}

@app.get("/async")
async def async_endpoint():
    return {"type": "async"}
```

- Use `async def` when your function `await`s something (an async database driver, `httpx` async client, etc.).
- Use plain `def` for CPU-bound or blocking work — FastAPI automatically runs these in a thread pool so they don't block the event loop.
- **Never** call blocking code (e.g. `time.sleep`, a synchronous DB driver) inside an `async def` function — that blocks the entire event loop for all concurrent requests.

10.2 Calling External APIs Asynchronously

```
import httpx

@app.get("/weather")
async def get_weather():
    async with httpx.AsyncClient() as client:
        response = await client.get("https://api.example.com/weather")
    return response.json()
```

10.3 Running Concurrent Tasks

```
import asyncio

@app.get("/aggregate")
async def aggregate():
    async with httpx.AsyncClient() as client:
        results = await asyncio.gather(
            client.get("https://api.example.com/a"),
            client.get("https://api.example.com/b"),
        )
    return [r.json() for r in results]
```

`asyncio.gather` runs both requests concurrently rather than sequentially, significantly reducing total latency for I/O-bound work.

Chapter 11: Background Tasks

For work that should happen *after* a response is returned — sending a confirmation email, writing a log — use `BackgroundTasks` rather than blocking the client:

```
from fastapi import BackgroundTasks

def write_log(message: str):
    with open("log.txt", "a") as f:
        f.write(message + "\n")

@app.post("/send-notification/{email}")
def send_notification(email: str, background_tasks: BackgroundTasks):
    background_tasks.add_task(write_log, f"notification sent to {email}")
    return {"message": "Notification sent in the background"}
```

For heavier or longer-running jobs (video processing, large batch emails), prefer a dedicated task queue like **Celery** or **arq** backed by Redis, rather than `BackgroundTasks`, which runs in-process and isn't durable across restarts.

Chapter 12: WebSockets

FastAPI supports WebSockets for real-time, bidirectional communication.

```
from fastapi import WebSocket, WebSocketDisconnect

class ConnectionManager:
    def __init__(self):
        self.active_connections: list[WebSocket] = []

    async def connect(self, websocket: WebSocket):
        await websocket.accept()
        self.active_connections.append(websocket)

    def disconnect(self, websocket: WebSocket):
        self.active_connections.remove(websocket)

    async def broadcast(self, message: str):
        for connection in self.active_connections:
            await connection.send_text(message)

manager = ConnectionManager()

@app.websocket("/ws/{client_id}")
async def websocket_endpoint(websocket: WebSocket, client_id: int):
    await manager.connect(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            await manager.broadcast(f"Client #{client_id}: {data}")
    except WebSocketDisconnect:
        manager.disconnect(websocket)
        await manager.broadcast(f"Client #{client_id} left the chat")
```

Each connected client keeps an open TCP connection; the server can push messages at any time without the client polling.

Chapter 13: File Uploads and Forms

13.1 Uploading a Single File

```
from fastapi import File, UploadFile

@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    contents = await file.read()
    return {"filename": file.filename, "size": len(contents)}
```

`UploadFile` streams the file to disk in chunks under the hood, making it efficient for large files, and exposes metadata like `content_type`.

13.2 Multiple Files

```
@app.post("/upload-multiple")
async def upload_multiple(files: list[UploadFile] = File(...)):
    return {"filenames": [f.filename for f in files]}
```

13.3 Form Data

```
from fastapi import Form

@app.post("/login")
def login(username: str = Form(...), password: str = Form(...)):
    return {"username": username}
```

Install `python-multipart` to enable form and file parsing:

```
pip install python-multipart
```


Chapter 14: Testing FastAPI Applications

14.1 The Test Client

FastAPI's `TestClient` (built on `httpx`) lets you call your API in-process, without running a server:

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_read_main():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Hello, FastAPI!"}
```

14.2 Testing POST Requests with a Body

```
def test_create_item():
    response = client.post("/items", json={"name": "Foo", "price": 9.99})
    assert response.status_code == 201
    data = response.json()
    assert data["name"] == "Foo"
```

14.3 Overriding Dependencies in Tests

You can swap real dependencies (a live database) for fakes during testing:

```
def override_get_db():
    db = TestingSessionLocal()
    try:
        yield db
    finally:
        db.close()

app.dependency_overrides[get_db] = override_get_db
```

14.4 Testing Async Endpoints

```
import pytest
from httpx import AsyncClient, ASGITransport

@pytest.mark.asyncio
async def test_async_endpoint():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        response = await ac.get("/async")
    assert response.status_code == 200
```

14.5 Running Tests

```
pip install pytest pytest-asyncio
pytest -v
```

Chapter 15: Advanced Pydantic

15.1 Custom Validators

```
from pydantic import BaseModel, field_validator

class User(BaseModel):
    username: str
    password: str

    @field_validator("password")
    @classmethod
    def password_must_be_strong(cls, v: str) -> str:
        if len(v) < 8:
            raise ValueError("Password must be at least 8 characters")
        return v
```

15.2 Model Config and Aliases

```
from pydantic import BaseModel, ConfigDict, Field

class Item(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    item_name: str = Field(alias="itemName")
```

This lets your API accept `camelCase` JSON from a frontend while keeping `snake_case` in your Python code.

15.3 Settings Management with `pydantic-settings`

```
pip install pydantic-settings
```

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    app_name: str = "My API"
    database_url: str
    debug: bool = False

    class Config:
        env_file = ".env"

settings = Settings()
```

This reads configuration from environment variables (or a `.env` file), validating types automatically — the standard way to manage config in production FastAPI apps.

15.4 Computed Fields and Model Methods

```
from pydantic import computed_field

class Rectangle(BaseModel):
    width: float
    height: float

    @computed_field
    @property
    def area(self) -> float:
        return self.width * self.height
```

Chapter 16: Project Structure and Best Practices

16.1 A Scalable Directory Layout

For anything beyond a toy app, split routes, models, and logic into modules using an `APIRouter` :

```
app/
├── main.py
├── core/
│   ├── config.py
│   └── security.py
├── models/
│   └── user.py
├── schemas/
│   └── user.py
├── routers/
│   ├── users.py
│   └── items.py
└── services/
    └── user_service.py
```

16.2 `APIRouter`

```
# routers/users.py
from fastapi import APIRouter

router = APIRouter(prefix="/users", tags=["users"])

@router.get("/")
def list_users():
    return []

@router.get("/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}
```

```
# main.py
from fastapi import FastAPI
from routers import users, items

app = FastAPI()
app.include_router(users.router)
app.include_router(items.router)
```

16.3 Best Practices Checklist

- Keep path operation functions thin; push business logic into a service layer.
- Use `response_model` on every endpoint to control your public contract.
- Centralize configuration with `pydantic-settings` , never hardcode secrets.
- Version your API (e.g. `/api/v1/...`) from day one.
- Use dependency injection for anything cross-cutting: auth, DB sessions, rate limits.
- Write tests alongside routes, not after the fact.
- Log structured JSON in production, not plain `print()` .

Chapter 17: Deployment

17.1 Production ASGI Servers

Uvicorn is great for development. In production, run it behind **Gunicorn** with Uvicorn workers for process management and graceful restarts:

```
pip install gunicorn
gunicorn main:app --workers 4 --worker-class uvicorn.workers.UvicornWorker --bind 0.0.0.0:80
```

17.2 Dockerizing a FastAPI App

```
FROM python:3.12-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "80"]
```

```
docker build -t my-fastapi-app .
docker run -p 80:80 my-fastapi-app
```

17.3 Reverse Proxy with Nginx

In production, place Nginx in front of your app servers to handle TLS termination, static file serving, and load balancing across multiple Uvicorn/Gunicorn processes.

17.4 Health Checks and Observability

```
@app.get("/health")
def health_check():
    return {"status": "ok"}
```

Pair this with structured logging and a metrics exporter (e.g. `prometheus-fastapi-instrumentator`) so your orchestration platform (Kubernetes, ECS) can monitor liveness and readiness.

Chapter 18: Performance and Advanced Topics

18.1 Caching

For expensive or frequently-repeated computations, cache results in Redis:

```
import redis
from fastapi import FastAPI

r = redis.Redis(host="localhost", port=6379, decode_responses=True)

@app.get("/expensive")
def expensive_operation():
    cached = r.get("expensive_result")
    if cached:
        return {"result": cached, "cached": True}
    result = compute_expensive_thing()
    r.set("expensive_result", result, ex=300) # expire after 5 minutes
    return {"result": result, "cached": False}
```

18.2 Rate Limiting

```
pip install slowapi
```

```
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter

@app.get("/limited")
@limiter.limit("5/minute")
def limited_endpoint(request: Request):
    return {"message": "This endpoint is rate-limited"}
```

18.3 Pagination

```
@app.get("/items")
def list_items(page: int = 1, page_size: int = 20):
    start = (page - 1) * page_size
    end = start + page_size
    return {"items": all_items[start:end], "page": page, "total": len(all_items)}
```

18.4 GraphQL Alongside REST (Strawberry)

FastAPI can also serve GraphQL via **Strawberry**, mounted as a sub-application, if your project needs both styles of API.

18.5 Lifespan Events

Use the `lifespan` context manager to run startup/shutdown logic (e.g. warming a connection pool):

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    print("Starting up...")
    yield
    print("Shutting down...")

app = FastAPI(lifespan=lifespan)
```

Appendix A: Quick Reference Cheat Sheet

Task	Snippet
Create app	<code>app = FastAPI()</code>
GET route	<code>@app.get("/path")</code>
POST route	<code>@app.post("/path")</code>
Path param	<code>def f(id: int):</code>
Query param	<code>def f(q: str = None):</code>
Request body	<code>def f(item: Item):</code> (Pydantic model)
Response model	<code>@app.get("/", response_model=Out)</code>
Status code	<code>@app.post("/", status_code=201)</code>
Dependency	<code>def f(x=Depends(dep)):</code>
Raise error	<code>raise HTTPException(404, "msg")</code>
Background task	<code>bg.add_task(fn, arg)</code>
WebSocket	<code>@app.websocket("/ws")</code>
Run dev server	<code>fastapi dev main.py</code>
Run prod server	<code>uvicorn main:app --workers 4</code>

Appendix B: Further Learning

- Official documentation: <https://fastapi.tiangolo.com>
- Source code: <https://github.com/fastapi/fastapi>
- Pydantic documentation: <https://docs.pydantic.dev>
- Starlette documentation: <https://www.starlette.io>
- SQLAlchemy documentation: <https://sqlmodel.tiangolo.com>

Closing Notes

FastAPI rewards learning Python's type system well — the better you understand type hints, the more the framework does for you automatically. Build small projects as you go: a to-do API, then add auth, then a database, then tests, then deploy it. Each chapter in this book maps to a milestone you can reach in a weekend.

Happy building!